CALL CENTER INTERNACIONAL – DECISIONES DE DISEÑO

YEIMMY JOHANA JIMENEZ YAYA

UNIVERSIDAD DE MANIZALES
FACULTAD DE CIENCIAS E INGENIERÍA
2026

1. Introducción

En este documento explico cómo diseñé y construí un sistema de información para un call center internacional, usando programación orientada a objetos en Python. El problema que resuelve es el siguiente: en un call center llegan llamadas de clientes de distintos países, y cada llamada hay que asignarla a un agente que esté libre y hable el idioma del cliente, atenderla y dejar un registro de cómo terminó. El propósito del sistema es organizar ese proceso y controlar las situaciones en las que algo sale mal, por ejemplo, que el agente esté ocupado o que la cola de espera ya no tenga espacio.

2. Descripción general de la solución

El proyecto está organizado en tres partes: la carpeta models, con las clases del sistema; la carpeta controllers, con la clase CentroLlamadas; y el archivo main.py, que ejecuta las pruebas. El centro de llamadas guarda los agentes y una cola de espera. Cada llamada une a un cliente con un agente y tiene su propio registro. Hay tres tipos de agente (soporte, ventas y cobranza) que comparten una misma base, y el sistema define tres excepciones propias para los casos anormales. El diagrama UML de clases está en la carpeta Diagramas del repositorio.

3. Desarrollo de las decisiones de diseño

3.1 El problema y las clases principales

Para representar el call center definí nueve clases. Agente es la clase abstracta que guarda lo común a todos los agentes: id, nombre, idioma, zona horaria y si está disponible. De ella heredan AgenteSoporte, AgenteVentas y AgenteCobranza. Cliente guarda los datos de quien llama (id, nombre, país, idioma y teléfono); su idioma es el dato clave para decidir qué agente puede atenderlo. Llamada representa cada llamada, con su código, el cliente, el motivo, el estado y el agente asignado; es la responsable de asignar un agente (asignar_agente()) y de terminarla (finalizar_llamada()). RegistroLlamada guarda cómo terminó la llamada: duración, resultado y observaciones. Por último, CentroLlamadas, que está en la carpeta controllers, guarda la lista de agentes y la cola de espera, y se encarga de buscar un agente disponible (buscar_agente_disponible()) y de agregar llamadas a la cola (agregar_a_cola()). A esto se suman las excepciones, que están en models/excepciones.py.

3.2 La clase abstracta Agente

Agente se definió como clase abstracta (hereda de ABC) porque en el call center no existe un agente genérico: siempre es de soporte, de ventas o de cobranza. Por eso no debe instanciarse. Además, hay cosas que no se pueden definir sin saber el tipo de agente: cómo atiende una llamada y cómo se calcula su bono. Para eso Agente declara dos métodos abstractos, atender_llamada() y calcular_bono(), que obligan a cada clase hija a escribir su propia versión. En main.py comprobé esto intentando crear un Agente directamente: Python lanza un TypeError, que se captura con try/except y el programa sigue. La clase abstracta también cumple una función real porque permite guardar los tres tipos de agente en una sola lista y tratarlos de la misma forma.

3.3 Herencia: una relación “es un”

AgenteSoporte, AgenteVentas y AgenteCobranza heredan de Agente porque un agente de soporte, un agente de ventas y un agente de cobranza son, cada uno, un agente. No usé herencia por obligación: los tres comparten los mismos datos y comportamientos básicos, pero cada uno hace algo distinto.

Los atributos comunes, que están en Agente y las hijas reciben con super().__init__(), son id_agente, nombre, idioma, zona_horaria y disponible. Los métodos comunes son los get_ de esos datos, esta_disponible(), set_nombre(), set_idioma(), set_zona_horaria(), set_disponible() y mostrar_informacion(). Todos los atributos son privados, y solo se leen o cambian con los get_ y set_, que validan que no queden vacíos.

Lo propio de cada clase hija es lo siguiente:

Clase hija	Atributos propios	Métodos propios
AgenteSoporte	nivel_soporte, tickets_resueltos	registrar_ticket(), get_nivel_soporte(), set_nivel_soporte()
AgenteVentas	meta_mensual, ventas_cerradas	registrar_venta(), get_meta_mensual(), set_meta_mensual()
AgenteCobranza	monto_recuperado, porcentaje_comision	registrar_pago(monto), set_porcentaje_comision()

3.4 Sobrescritura y polimorfismo

Las tres clases hijas sobrescriben tres métodos de Agente: atender_llamada(), calcular_bono() y mostrar_informacion(). El nombre es el mismo, pero el resultado

cambia según el tipo. En atender_llamada(), el agente de soporte revisa el problema del cliente, el de ventas presenta una oferta y el de cobranza recuerda un pago pendiente. En calcular_bono(), soporte recibe $5.000 por cada ticket resuelto; ventas recibe $20.000 por cada venta y $100.000 más si cumple la meta mensual; y cobranza recibe un porcentaje del monto recuperado. En mostrar_informacion(), cada hija llama primero al método de Agente con super() y luego agrega sus datos propios.
El polimorfismo se ve al final de main.py. Creé una sola lista, agentes, con agentes de los tres tipos, y la recorro con un único ciclo for que llama siempre a mostrar_informacion(), atender_llamada() y calcular_bono(). Python ejecuta la versión que le corresponde a cada objeto, sin que yo use condicionales para preguntar de qué tipo es. El único if de main.py está en la prueba de buscar un agente que hable francés, y no se usa para decidir qué método ejecutar según el tipo.

3.5 Composición y otras relaciones entre objetos

La composición está entre Llamada y RegistroLlamada. La llamada crea su propio registro dentro de su constructor (self.__registro = RegistroLlamada()), y el constructor del registro no recibe datos: nace vacío y se completa cuando la llamada termina, a través de registrar_cierre(), que se llama desde finalizar_llamada(). Es composición porque el registro solo tiene sentido como parte de una llamada y no existe por su cuenta: una llamada tiene un registro. No lo resolví con herencia porque un registro no es un tipo de llamada ni una llamada es un tipo de registro; la herencia se usa cuando algo es un tipo de otra cosa, como un agente de soporte es un agente.
Además hay otras relaciones entre objetos. Llamada recibe un Cliente ya creado en su constructor, y asignar_agente() recibe un Agente ya creado, así que ni el cliente ni el agente dependen de la llamada para existir. CentroLlamadas guarda una lista de agentes y una cola de llamadas, también creados por fuera.

3.6 Excepciones personalizadas

Definí una excepción base, CallCenterError, y tres excepciones propias que heredan de ella. Cada una se lanza con raise en el método donde se detecta el problema y se captura con try/except en main.py, donde se muestra el mensaje y el programa continúa.

Excepción	Situación que controla	Dónde se lanza
AgenteNoDisponibleError	Se asigna una llamada a un agente que ya está ocupado	Llamada.asignar_agente()
IdiomaNoSoportadoError	El agente no habla el idioma del cliente	Llamada.asignar_agente()
ColaLlenaError	La cola de espera ya no admite más llamadas	CentroLlamadas.agregar_a_cola()

En las pruebas, cuando fallan las dos primeras, la llamada se manda a la cola de espera, y cuando la cola se llena, el programa muestra el mensaje y sigue ejecutándose hasta el final.

3.7 Decisiones que cambiaron durante el desarrollo

Primero tuve que elegir el contexto: consideré un banco y un call center, y elegí el call center internacional porque el idioma y la zona horaria permitían diferenciar bien a los agentes y a las llamadas. Después, el primer diagrama UML tenía solo los atributos y métodos principales, y lo completé con los constructores, los get_ y set_ y los tipos de datos, para que coincidiera con el código. También ajusté RegistroLlamada: agregué registrar_cierre() para que el registro naciera vacío dentro de la llamada y se completara cuando esta termina.

3.8 Cómo verifiqué que el código funciona

Fui probando cada clase por separado en la terminal antes de subirla a GitHub, por ejemplo, comprobando que Agente no se puede instanciar y que el bono de cada tipo de agente da el valor esperado. Después ejecuté main.py completo y revisé que salieran los casos normales, las tres excepciones capturadas, el mensaje de que el programa continúa y el recorrido polimórfico. También comparé el diagrama UML con las clases, atributos y métodos del código, y subí el trabajo con commits por cada avance.

4. Conclusiones

Con este proyecto entendí cómo se combinan los conceptos de la programación orientada a objetos en un mismo sistema: la clase abstracta obliga a las hijas a tener ciertos métodos, la herencia evita repetir código, la sobrescritura hace que cada tipo se comporte distinto, la composición se usa cuando una clase tiene como parte esencial a otra, y las excepciones permiten que los errores se expliquen y el programa no se detenga. Las dificultades más grandes fueron manejar Git y GitHub (comandos,

carpetas y etiquetas) y entender cuándo usar composición y cuándo herencia. Lo que más me ayudó fue pensar en “es un” y “tiene un”.

5. Uso de herramientas de inteligencia artificial

Usé Claude, una herramienta de inteligencia artificial, como apoyo durante todo el proyecto. Lo usé para entender los conceptos de POO, para proponer opciones de contexto (banco y call center) y la lista de clases, para generar una base del código y del diagrama UML, y para resolver errores de Git y de ejecución. Probé y revisé el código, y lo comparé con el diagrama para poder explicarlo.

De lo que propuso la herramienta, descarté la opción del banco y elegí el call center; pedí que el diagrama quedara más completo, con constructores, getters, setters y tipos; y revisé y cambié los datos de ejemplo y los mensajes de los programas para dejarlos a mi manera.

6. Herramientas consultadas

Claude (Anthropic), como apoyo con IA; draw.io, para el diagrama UML de clases; Python 3, para el desarrollo; Visual Studio Code, como editor; y Git y GitHub, para el control de versiones y la entrega.